Experiment 'p_14_2_selection_sort_interval_seq' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_14_2_selection_sort_interval_seq
Natural language statement: null
Method signature: p_14_2_selection_sort_interval_seq(inputs: seq<int>, bound: nat) returns (sorted: seq<int>)

Ensures

- `forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]`
- `multiset(inputs) == multiset(sorted)`

Requires

- `forall i :: 0 <= i < |inputs| ==> 0 <= inputs[i] <= bound`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_14_2_selection_sort_interval_seq(inputs: seq<int>, bound: nat) returns (sorted: seq<int
    ↪ >)

The method should respect the following contract:

requires forall i :: 0 <= i < |inputs| ==> 0 <= inputs[i] <= bound, ensures forall i, j
    ↪ :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j], ensures multiset(inputs) ==
    ↪ multiset(sorted)

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnPjNaeSagIF9yBwJgmczeyIaXgE4
Model: gpt-5.5-2026-04-23
Created at: 1780669353
Finish reason: stop
Usage: 358 tokens in, and 7439 tokens out

```
{
  sorted := [];
  var rest := inputs;
  var v: int := 0;

  while v <= bound
    invariant 0 <= v <= bound + 1
    invariant multiset(inputs) == multiset(sorted) + multiset(rest)
    invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
    invariant forall i :: 0 <= i < |sorted| ==> sorted[i] < v
    invariant forall i :: 0 <= i < |rest| ==> v <= rest[i] <= bound
    decreases bound + 1 - v
  {
    assert 0 <= v <= bound;

    while exists k :: 0 <= k < |rest| && rest[k] == v
      invariant 0 <= v <= bound
      invariant multiset(inputs) == multiset(sorted) + multiset(rest)
```

```
  invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
  invariant forall i :: 0 <= i < |sorted| ==> sorted[i] <= v
  invariant forall i :: 0 <= i < |rest| ==> v <= rest[i] <= bound
  decreases |rest|
{
  var k :| 0 <= k < |rest| && rest[k] == v;
  var oldSorted := sorted;
  var oldRest := rest;
  var newSorted := oldSorted + [v];
  var newRest := oldRest[..k] + oldRest[(k + 1)..];

  assert oldRest == oldRest[..k] + [oldRest[k]] + oldRest[(k + 1)..];
  assert oldRest[k] == v;
  assert multiset(oldRest) == multiset([v]) + multiset(newRest);
  assert multiset(newSorted) == multiset(oldSorted) + multiset([v]);
  calc {
    multiset(inputs);
    == multiset(oldSorted) + multiset(oldRest);
    == multiset(oldSorted) + (multiset([v]) + multiset(newRest));
    == (multiset(oldSorted) + multiset([v])) + multiset(newRest);
    == multiset(newSorted) + multiset(newRest);
  }

  assert forall a, b :: 0 <= a < b < |newSorted| ==> newSorted[a] <= newSorted[b] by {
    forall a, b | 0 <= a < b < |newSorted|
      ensures newSorted[a] <= newSorted[b]
    {
      if b < |oldSorted| {
        assert newSorted[a] == oldSorted[a];
        assert newSorted[b] == oldSorted[b];
      } else {
        assert b == |oldSorted|;
        assert a < |oldSorted|;
        assert newSorted[a] == oldSorted[a];
        assert newSorted[b] == v;
        assert oldSorted[a] <= v;
      }
    }
  }

  assert forall t :: 0 <= t < |newSorted| ==> newSorted[t] <= v by {
    forall t | 0 <= t < |newSorted|
      ensures newSorted[t] <= v
    {
      if t < |oldSorted| {
        assert newSorted[t] == oldSorted[t];
      } else {
        assert t == |oldSorted|;
        assert newSorted[t] == v;
      }
    }
```

```
    }

    assert forall t :: 0 <= t < |newRest| ==> v <= newRest[t] <= bound by {
      forall t | 0 <= t < |newRest|
        ensures v <= newRest[t] <= bound
      {
        if t < k {
          assert newRest[t] == oldRest[t];
        } else {
          assert t + 1 < |oldRest|;
          assert newRest[t] == oldRest[t + 1];
        }
      }
    }

    sorted := newSorted;
    rest := newRest;
  }

  assert !(exists k :: 0 <= k < |rest| && rest[k] == v);
  assert forall t :: 0 <= t < |rest| ==> v + 1 <= rest[t] <= bound by {
    forall t | 0 <= t < |rest|
      ensures v + 1 <= rest[t] <= bound
    {
      assert v <= rest[t] <= bound;
      assert rest[t] != v;
      assert v < rest[t];
    }
  }
  assert forall t :: 0 <= t < |sorted| ==> sorted[t] < v + 1 by {
    forall t | 0 <= t < |sorted|
      ensures sorted[t] < v + 1
    {
      assert sorted[t] <= v;
    }
  }
  v := v + 1;
 }

 assert v == bound + 1;
 if |rest| != 0 {
   assert 0 <= 0 < |rest|;
   assert v <= rest[0] <= bound;
   assert false;
 }
 assert |rest| == 0;
 assert rest == [];
 assert multiset(rest) == multiset{};
 assert multiset(inputs) == multiset(sorted);
}
```

Final Program

```
method p_14_2_selection_sort_interval_seq(inputs: seq<int>, bound: nat) returns (sorted:
    ↪ seq<int>)
      requires forall i :: 0 <= i < |inputs| ==> 0 <= inputs[i] <= bound
      ensures forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
      ensures multiset(inputs) == multiset(sorted)
{
  sorted := [];
  var rest := inputs;
  var v: int := 0;

  while v <= bound
    invariant 0 <= v <= bound + 1
    invariant multiset(inputs) == multiset(sorted) + multiset(rest)
    invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
    invariant forall i :: 0 <= i < |sorted| ==> sorted[i] < v
    invariant forall i :: 0 <= i < |rest| ==> v <= rest[i] <= bound
    decreases bound + 1 - v
  {
    assert 0 <= v <= bound;

    while exists k :: 0 <= k < |rest| && rest[k] == v
      invariant 0 <= v <= bound
      invariant multiset(inputs) == multiset(sorted) + multiset(rest)
      invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
      invariant forall i :: 0 <= i < |sorted| ==> sorted[i] <= v
      invariant forall i :: 0 <= i < |rest| ==> v <= rest[i] <= bound
      decreases |rest|
    {
      var k :| 0 <= k < |rest| && rest[k] == v;
      var oldSorted := sorted;
      var oldRest := rest;
      var newSorted := oldSorted + [v];
      var newRest := oldRest[..k] + oldRest[(k + 1)..];

      assert oldRest == oldRest[..k] + [oldRest[k]] + oldRest[(k + 1)..];
      assert oldRest[k] == v;
      assert multiset(oldRest) == multiset([v]) + multiset(newRest);
      assert multiset(newSorted) == multiset(oldSorted) + multiset([v]);
      calc {
        multiset(inputs);
        == multiset(oldSorted) + multiset(oldRest);
        == multiset(oldSorted) + (multiset([v]) + multiset(newRest));
        == (multiset(oldSorted) + multiset([v])) + multiset(newRest);
        == multiset(newSorted) + multiset(newRest);
      }

      assert forall a, b :: 0 <= a < b < |newSorted| ==> newSorted[a] <= newSorted[b] by {
        forall a, b | 0 <= a < b < |newSorted|
          ensures newSorted[a] <= newSorted[b]
        {
```

```
      if b < |oldSorted| {
        assert newSorted[a] == oldSorted[a];
        assert newSorted[b] == oldSorted[b];
      } else {
        assert b == |oldSorted|;
        assert a < |oldSorted|;
        assert newSorted[a] == oldSorted[a];
        assert newSorted[b] == v;
        assert oldSorted[a] <= v;
      }
    }
  }

  assert forall t :: 0 <= t < |newSorted| ==> newSorted[t] <= v by {
    forall t | 0 <= t < |newSorted|
      ensures newSorted[t] <= v
    {
      if t < |oldSorted| {
        assert newSorted[t] == oldSorted[t];
      } else {
        assert t == |oldSorted|;
        assert newSorted[t] == v;
      }
    }
  }

  assert forall t :: 0 <= t < |newRest| ==> v <= newRest[t] <= bound by {
    forall t | 0 <= t < |newRest|
      ensures v <= newRest[t] <= bound
    {
      if t < k {
        assert newRest[t] == oldRest[t];
      } else {
        assert t + 1 < |oldRest|;
        assert newRest[t] == oldRest[t + 1];
      }
    }
  }

  sorted := newSorted;
  rest := newRest;
}

assert !(exists k :: 0 <= k < |rest| && rest[k] == v);
assert forall t :: 0 <= t < |rest| ==> v + 1 <= rest[t] <= bound by {
  forall t | 0 <= t < |rest|
    ensures v + 1 <= rest[t] <= bound
  {
    assert v <= rest[t] <= bound;
    assert rest[t] != v;
    assert v < rest[t];
```

```
    }
   }
   assert forall t :: 0 <= t < |sorted| ==> sorted[t] < v + 1 by {
    forall t | 0 <= t < |sorted|
      ensures sorted[t] < v + 1
    {
      assert sorted[t] <= v;
    }
   }
   v := v + 1;
  }

  assert v == bound + 1;
  if |rest| != 0 {
   assert 0 <= 0 < |rest|;
   assert v <= rest[0] <= bound;
   assert false;
  }
  assert |rest| == 0;
  assert rest == [];
  assert multiset(rest) == multiset{};
  assert multiset(inputs) == multiset(sorted);
}
```

Total Token Usage

Input tokens: 358
Output tokens: 7439
Reasoning tokens: 6143
Sum of 'total tokens': 7797

Experiment Timings

Overall Experiment started at 1780669352432, ended at 1780669500966, lasting 148534ms (148.53 seconds)
Iteration #1 started at 1780669352458, ended at 1780669500966, lasting 148508ms (148.51 seconds)